Micro Goal-Secure Goal Management Backend API

(Team Id: SWTID-2026-8352)

Introduction

Project Title: Micro Goal

Team ID: (Team Id: SWTID-2026-8352)

Team Leader:

Shaik Farhan & shaikfarhin78@gmail.com

Team Members:

Harini R & hr0629158@gmail.com

Sharmila S & sharmikalpana@gmail.com

Jeeva & jeevacap751@gmail.com

Project Overview

Micro Goals is a secure backend RESTful API developed using **Node.js**, **Express.js**, and **MongoDB**. The system is designed to help users manage their personal goals digitally in a secure and efficient way. It provides functionalities such as user registration, authentication, and goal management through structured API endpoints.

The application follows the **RESTful architecture**, which uses standard HTTP methods such as **GET, POST, PUT, and DELETE** to perform operations on data. This architecture makes the system scalable, organized, and easy to integrate with different frontend applications like web or mobile apps.

The backend server handles all the important tasks such as processing requests, validating user data, storing information in the database, and sending responses to the client. To ensure security, the system implements **JSON Web Token (JWT)** authentication along with middleware functions that protect sensitive routes and verify user identity.

The system also uses a centralized error-handling mechanism which helps manage and respond to errors in a consistent way across the entire application. This improves the reliability and maintainability of the backend system.

Key Features of the System

1. User Registration and Login

The system allows users to create an account by providing necessary details such as name, email, and password. During registration, the data is validated and stored securely in the database.

Once registered, users can log in using their credentials. After successful authentication, the server verifies the user and allows access to the system. This feature ensures that only registered users can use the application.

2. JWT-Based Authentication

To maintain secure communication between the client and server, the system uses **JSON Web Token (JWT)** authentication. After a user logs in successfully, the server generates a unique token.

This token is sent to the client and must be included in future requests to access protected resources. The server verifies the token before processing the request, ensuring that the user is authenticated and authorized.

3. CRUD Operations for Goals

The application allows users to manage their goals using **CRUD operations**. CRUD stands for:

- **Create:** Users can add new personal goals.

- **Read:** Users can view the list of their goals.
- **Update:** Users can edit or modify their existing goals.
- **Delete:** Users can remove goals that are completed or no longer needed.

These operations allow users to organize and track their progress effectively.

4. Protected Routes

Some API routes are restricted and can only be accessed by authenticated users. These routes are protected using authentication middleware.

When a request is made to a protected route, the middleware checks whether a valid JWT token is present. If the token is valid, the request is processed. Otherwise, the system denies access. This ensures that users can only access their own data and prevents unauthorized access.

5. Middleware for Security and Request Handling

Middleware functions play an important role in the backend system. They act as intermediate functions that process requests before reaching the final route handler.

In this project, middleware is used for:

- Authenticating users
- Validating tokens
- Handling request data
- Managing errors

This helps maintain a clean and organized backend structure.

6. Centralized Error Handling

The system includes centralized error handling to manage errors effectively. Instead of writing separate error-handling logic for every route, a single error-handling middleware is used.

This middleware captures errors that occur during request processing and sends a proper response to the client. It improves debugging, ensures consistent responses, and enhances the overall reliability of the system.

Project Instructions

The **Micro Goals** project is developed by following standard backend development practices and modern web application design principles. The system is designed to be secure, scalable, and easy to maintain. Several technologies and architectural patterns are used to organize the project structure and ensure efficient data handling.

1. REST API Principles

The Micro Goals backend follows **REST API principles** to ensure structured communication between the client and the server. REST (Representational State Transfer) is an architectural style that uses standard HTTP methods to perform operations on resources.

In this project, different HTTP methods are used to handle goal management operations:

- **GET** – Used to retrieve data from the server, such as viewing user goals.
- **POST** – Used to create new data, such as registering a new user or adding a new goal.
- **PUT** – Used to update existing data, such as editing a goal.
- **DELETE** – Used to remove data from the database, such as deleting a goal.

By following REST principles, the API becomes organized, predictable, and easy for developers to understand and integrate with frontend applications.

2. Authentication Using JWT

User authentication in the project is implemented using **JSON Web Token (JWT)**. JWT is a secure method for verifying the identity of users in web applications.

When a user logs in successfully, the server generates a JWT token. This token contains encoded information about the user and is sent back to the client. The client must include this token in the header of future API requests.

Before processing a request, the server verifies the token using authentication middleware. If the token is valid, the request is allowed to proceed. Otherwise, the system denies access.

JWT authentication helps ensure that only authorized users can access protected routes and manage their personal goals securely.

3. Database Using MongoDB

The project uses **MongoDB** as the primary database to store user information and goal data. MongoDB is a NoSQL database that stores data in a flexible **document-based format (JSON-like structure)**.

In this system, MongoDB is used to store:

- User details such as name, email, and password
- User-created goals
- Goal information such as title, description, and completion status

MongoDB is chosen because it provides high performance, flexibility, and scalability. It integrates well with backend technologies like **Node.js** and **Express.js**, making it a popular choice for modern web applications.

4. API Testing Using Thunder Client

All API endpoints in the Micro Goals project are tested using **Thunder Client**, which is a lightweight REST API testing tool available as an extension in **Visual Studio Code**.

Thunder Client allows developers to send HTTP requests to the server and view responses directly within the editor. It helps verify whether the API endpoints are working correctly.

Using Thunder Client, developers can test:

- User registration APIs
- Login authentication APIs
- Goal creation APIs
- Goal update APIs
- Goal deletion APIs
- Protected routes with JWT tokens

Testing APIs during development ensures that the backend functions correctly before integrating it with a frontend application.

5. MVC Architecture

The Micro Goals project follows the **MVC (Model–View–Controller) architecture** to maintain a clean and organized project structure.

MVC separates the application into three main components:

Model
The Model represents the database structure and handles data-related operations. In this project, models define the structure of users and goals stored in MongoDB.

View
The View represents the user interface. Since this project focuses mainly on the backend API, the view is handled by the client application that consumes the API.

Controller
The Controller manages the application logic. It processes incoming requests, interacts with the models to retrieve or modify data, and sends responses back to the client.

Using MVC architecture helps in:

- Organizing the code structure
- Improving maintainability
- Making the application easier to scale
- Separating logic from data handling

Pre-requisites

Before running and working with the Micro Goals project, certain tools and basic technical knowledge are required. These prerequisites help ensure that the development environment is properly set up and that the backend API can run smoothly. Each tool listed below plays an important role in building, running, testing, and maintaining the Micro Goals backend system.

Node.js (v18 or higher)

Node.js is a JavaScript runtime environment that allows developers to run JavaScript code outside of a web browser. It is widely used for building scalable and efficient server-side applications. In the Micro Goals project, Node.js is responsible for executing the backend code and handling server operations such as processing requests, managing API routes, and communicating with the database. Node.js also includes **npm (Node Package Manager)**, which is used to install and manage project dependencies like Express.js, JWT libraries, database connectors, and other required packages. Using Node.js version 18 or higher ensures compatibility with modern JavaScript features and improved performance.

Express.js

Express.js is a lightweight and flexible web application framework built on top of Node.js. It simplifies the process of creating web servers and RESTful APIs. In the Micro Goals project, Express.js is used to define API routes for user authentication and goal management. It also helps manage HTTP requests and responses, handle middleware functions, and organize the server-side logic efficiently. Express.js allows developers to build APIs quickly while maintaining a clean and structured codebase.

MongoDB (Local / Atlas)

MongoDB is the database used in the Micro Goals project. It is a NoSQL, document-oriented database that stores data in flexible JSON-like documents. Unlike traditional relational databases, MongoDB does not require a fixed table structure, which allows developers to store and modify data easily. In this project, MongoDB is used to store user account information and goal-related data. Developers can use **MongoDB locally**, by installing it on their system, or they can use **MongoDB Atlas**, which is a cloud-based database service that allows remote access and automatic scaling. MongoDB integrates well with Node.js and is commonly used for modern web applications.

Visual Studio Code (VS Code)

Visual Studio Code is a powerful and widely used source code editor that supports multiple programming languages, including JavaScript. It provides useful features such as syntax highlighting, debugging tools, extensions, and an integrated terminal. In the Micro Goals project, VS Code is used to write, edit,

and manage the backend code. Developers can also run Node.js commands directly from the built-in terminal, install dependencies, and start the server from within the editor. Its extension support improves productivity and simplifies the development process.

Thunder Client (Extension)

Thunder Client is a REST API testing tool available as an extension in Visual Studio Code. It allows developers to send HTTP requests to the API endpoints and inspect the responses. Thunder Client supports different request methods such as GET, POST, PUT, and DELETE, which are essential for testing CRUD operations in the Micro Goals project. It also allows users to add request headers, authentication tokens, and JSON request bodies. By using Thunder Client, developers can verify whether the API endpoints for user registration, login, and goal management are functioning correctly before connecting the backend to a frontend application.

Basic Knowledge of JavaScript & REST APIs

To work with the Micro Goals project effectively, developers should have a basic understanding of JavaScript and REST APIs. Knowledge of JavaScript is important because the entire backend system is built using Node.js and Express.js. Developers should be familiar with concepts such as functions, objects, asynchronous programming, modules, and error handling. Additionally, understanding REST API principles is necessary for working with HTTP methods, creating API endpoints, sending requests, and handling JSON responses. This foundational knowledge helps developers understand how the system operates and makes it easier to maintain and extend the project.

Project Architecture (Text Representation)

The Micro Goals backend system follows a well-structured **project architecture** that ensures the application is scalable, secure, and easy to maintain. The architecture defines how different technologies and components interact with each other to handle requests, process business logic, store data, and provide secure access to users. By organizing the system into clearly

defined layers, developers can easily manage the codebase, debug issues, and extend the functionality of the application in the future.

The project is built using a **technical architecture** that includes a runtime environment, a web framework, a database system, an authentication mechanism, and an architectural design pattern. Each of these components plays a crucial role in the functioning of the Micro Goals backend API.

Runtime: Node.js

The runtime environment used in the Micro Goals project is **Node.js**. Node.js allows developers to run JavaScript code on the server side instead of the browser. It uses an event-driven and non-blocking input/output model, which makes it highly efficient for handling multiple client requests at the same time. This feature makes Node.js suitable for building scalable web applications and APIs. In the Micro Goals project, Node.js is responsible for executing the backend code, handling incoming HTTP requests, communicating with the database, and returning responses to the client application.

Framework: Express.js

The project uses **Express.js**, which is a lightweight and flexible web application framework built on top of Node.js. Express.js simplifies the development of RESTful APIs by providing tools for routing, middleware management, and request-response handling. It allows developers to define different API endpoints for operations such as user registration, login, and goal management. Express.js also supports middleware functions that help implement features like authentication, request validation, and error handling. In the Micro Goals application, Express.js acts as the main framework that manages the server and controls how requests are processed.

Database: MongoDB

The database used in this project is **MongoDB**, which is a NoSQL document-based database. MongoDB stores data in JSON-like documents rather than traditional tables used in relational databases. This flexible data structure allows developers to store complex data easily and modify the schema when needed. In the Micro Goals system, MongoDB stores user information and goal-related data. Each user and goal is stored as a document in separate collections. MongoDB works efficiently with Node.js through libraries such as Mongoose,

which helps define schemas, validate data, and perform database operations such as creating, updating, retrieving, and deleting records.

Authentication: JWT

To ensure secure access to the system, the project implements **JWT (JSON Web Token) authentication**. JWT is a token-based authentication method that allows the server to verify the identity of a user. When a user logs in successfully, the server generates a token that contains encoded information about the user. This token is sent to the client and must be included in the authorization header for all future requests to protected routes. Middleware in the application verifies the token before allowing access to the requested resource. If the token is missing, invalid, or expired, the request is denied. This mechanism ensures that only authenticated users can access sensitive operations such as creating, updating, or deleting goals.

Architecture Pattern: MVC

The Micro Goals project follows the **MVC (Model–View–Controller) architecture pattern**, which is widely used for organizing application code into separate layers. This design pattern helps maintain a clear separation of responsibilities within the system.

The **Model** layer represents the structure of the data and manages interactions with the MongoDB database. Models define schemas that describe how data such as users and goals are stored and validated.

The **Controller** layer contains the business logic of the application. Controllers handle incoming requests, process data, interact with the models, and send appropriate responses back to the client.

The **View** layer typically represents the user interface in traditional applications. However, in the Micro Goals backend API, the view is represented by **JSON responses** that are returned to the client application. These responses can later be used by a frontend application such as a web or mobile interface.

By using the MVC architecture, the project becomes easier to maintain, as changes in one component do not directly affect others. This separation also improves code readability, testing, and scalability.

ER Diagram

The Entity Relationship (ER) Diagram for the Micro Goals project represents the structure of the database and illustrates how different data entities are related to each other. In this system, the database mainly consists of two primary entities: **User** and **Goal**. These entities define how user information and goal-related data are stored and connected within the MongoDB database.

The first entity is the **User** entity, which represents individuals who register and use the Micro Goals application. Each user has a unique record in the database. The attributes of the User entity include **_id**, **name**, **email**, and **password**. The **_id** field is a unique identifier automatically generated by MongoDB for every user document. The **name** attribute stores the user's full name, which helps identify the account owner. The **email** attribute stores the user's email address and is typically used as a unique login credential for authentication. The **password** attribute stores the user's encrypted password, which is secured using hashing techniques to protect sensitive information and ensure safe authentication.

The second entity is the **Goal** entity, which represents the goals created and managed by users within the system. Each goal contains several attributes, including **_id**, **title**, **completed**, and **user Id**. Similar to the User entity, the **_id** field uniquely identifies each goal record in the database. The **title** attribute stores the description or name of the goal created by the user. The **completed** attribute is a Boolean value that indicates whether the goal has been achieved or not, allowing users to track their progress. The **user Id** attribute is a reference field that links the goal to the specific user who created it. This reference is important because it ensures that each goal belongs to a particular user in the system.

The relationship between the User and Goal entities is defined as a **one-to-many relationship**. This means that **one user can create and manage multiple goals**, but each goal belongs to only one user. In other words, a single user record in the User collection can be associated with multiple goal records in the Goal collection. This relationship is implemented in the database using the **user Id reference field** in the Goal entity, which stores the unique identifier of the user who owns the goal.

This ER structure helps maintain proper data organization and ensures that goal data is securely associated with the correct user. It also supports efficient data retrieval, as the system can easily fetch all goals related to a specific user by using the user's unique identifier. Overall, the ER diagram provides a clear

representation of how user accounts and goals are structured and connected within the Micro Goals database system.

Below is a **complete documentation-style explanation** covering **all feature points clearly in separate paragraphs**.

Features

The Micro Goals backend API provides several important features that allow users to securely create and manage their personal goals. These features ensure that the system is efficient, secure, and user-focused. The application is designed so that users can register, authenticate themselves, and perform goal management operations while ensuring that their data remains private and protected. The following features describe the main capabilities of the system.

User Registration

The **User Registration** feature allows new users to create an account in the MicroGoals system. During the registration process, users provide details such as their name, email address, and password. The system validates the provided information to ensure that the required fields are filled and that the email address is unique. For security purposes, the user's password is encrypted using hashing techniques before being stored in the database. This ensures that even if the database is accessed by unauthorized individuals, the original password cannot be easily retrieved. Once the registration is successful, the user can proceed to log in and access the system.

Secure Login with JWT

The **Secure Login with JWT (JSON Web Token)** feature enables registered users to authenticate themselves and access the system securely. When a user enters their email and password during login, the server verifies the credentials with the data stored in the database. If the credentials are valid, the server generates a JWT token that contains encoded user information. This token is returned to the client and must be included in the authorization header for future API requests. JWT-based authentication ensures that the system can securely verify user identity without maintaining server-side sessions.

Create Goal

The **Create Goal** feature allows authenticated users to add new goals to the system. After successful login, users can send a request to create a new goal by providing information such as the goal title or description. The system processes the request and stores the goal in the database along with a reference to the user who created it. This ensures that each goal is properly linked to its owner and can be managed individually.

Update Goal

The **Update Goal** feature allows users to modify existing goals. Users can update information such as the goal title or change the completion status of the goal. For example, a user can mark a goal as completed once it has been achieved. Before allowing the update operation, the system verifies that the goal belongs to the authenticated user. This prevents users from modifying goals created by other users.

Delete Goal

The **Delete Goal** feature allows users to remove goals that are no longer relevant or needed. When a delete request is sent, the system checks whether the authenticated user is the owner of that goal. If the verification is successful, the goal is permanently removed from the database. This functionality allows users to keep their goal list organized and updated.

View User-specific Goals

The **View User-specific Goals** feature ensures that users can only view the goals they have created. When a request is made to retrieve goals, the system filters the data using the user's unique identifier stored in the JWT token. As a result, only goals associated with that particular user are returned. This feature maintains data privacy and prevents users from accessing goals belonging to other users.

Protected Routes

The **Protected Routes** feature ensures that certain API endpoints are accessible only to authenticated users. These routes require a valid JWT token to be included in the request header. Middleware functions verify the authenticity and validity of the token before allowing the request to proceed. If the token is missing, invalid, or expired, the system rejects the request and returns an appropriate error message. Protected routes help safeguard sensitive operations such as creating, updating, or deleting goals.

Centralized Error Handling

The **Centralized Error Handling** feature improves the reliability and maintainability of the application. Instead of handling errors individually within each API route, the system uses a dedicated error-handling middleware. This middleware captures all errors generated in the application and returns a standardized response to the client. It ensures that users receive consistent error messages and proper HTTP status codes. Centralized error handling also helps developers easily identify and fix issues within the application.

Roles and Responsibilities

The MicroGoals project defines clear roles and responsibilities for users and developers to ensure proper usage, management, and development of the system. Each role has distinct tasks that contribute to the functionality, security, and maintenance of the backend API.

User

The **User** is the primary consumer of the MicroGoals application. Users are responsible for interacting with the system to manage their personal goals. Their responsibilities include:

- **Register**: Users create an account in the system by providing necessary details such as name, email, and password. Registration ensures that the user can securely access the platform.
- **Login**: Users authenticate themselves by providing valid credentials. Upon successful login, they receive a JWT token, which allows access to protected resources.
- **Manage Goals**: Users can create new goals, view existing goals, update goal details, and delete goals they no longer need. The system ensures that each user can only access their own goals, maintaining privacy and security.

By performing these actions, users leverage the core functionality of the Micro Goals system to track and achieve their personal objectives.

Developer

The **Developer** is responsible for building, maintaining, and testing the Micro Goals backend system. Their responsibilities include:

- **API Development**: Developers implement the backend using Node.js and Express.js, following RESTful API principles. They define routes, controllers, models, and middleware to handle user authentication, goal management, and security features.
- **Database Integration**: Developers design the MongoDB schema, create models for Users and Goals, and ensure that data is stored and retrieved efficiently.
- **Testing & Debugging**: Developers test API endpoints using tools like Thunder Client to ensure that all functionalities, including registration, login, and CRUD operations for goals, work as expected. They also debug any issues and optimize the code for better performance and reliability.
- **Security Implementation**: Developers implement JWT authentication, protected routes, and error-handling middleware to ensure that the application is secure and data integrity is maintained.
- **Documentation & Maintenance**: Developers maintain detailed documentation of the API, architecture, and project workflow. They also update the system as needed to add features, fix bugs, and ensure compatibility with new technologies.

By fulfilling these responsibilities, developers ensure that the Micro Goals system is robust, secure, and easy to maintain, providing a reliable backend service for users.

User Flow

The **User Flow** of the MicroGoals system describes the step-by-step process that a user follows to interact with the backend API. It illustrates how a user moves from registration to managing goals while ensuring secure access through authentication mechanisms. Understanding this flow is essential for documenting how the system operates from a user perspective.

1. User Registers

The first step in the system is **user registration**. A new user provides required information such as name, email, and password through the registration API endpoint. The system validates the provided data to ensure that all fields are

filled and that the email is unique. The password is securely hashed before being stored in the MongoDB database. After successful registration, the user account is created, allowing the user to access the system.

2. User Logs In

After registration, the user can **log in** using their registered email and password. The login endpoint verifies the credentials by comparing the submitted data with the records stored in the database. If the credentials are correct, the user is successfully authenticated; otherwise, an error message is returned.

3. JWT Token Generated

Upon successful login, the system generates a **JSON Web Token (JWT)**. This token contains encoded information about the user and serves as a secure method of authentication. The JWT is sent back to the client, and it must be included in all future requests to protected API endpoints. This token ensures that the server can identify and authorize the user without storing session data.

4. User Sends Token in Header

For accessing **protected routes**, the client includes the JWT token in the **authorization header** of every HTTP request. The format typically follows: Authorization: Bearer <token>. This step allows the server to verify the user's identity and confirm that the request is coming from an authenticated user.

5. Access Protected Routes

Once the JWT token is verified, the user gains access to **protected routes**. Protected routes are API endpoints that require authentication to perform operations, such as creating, updating, or deleting goals. Middleware on the server checks the token, validates the user, and ensures that only authorized users can access these resources.

6. Perform CRUD Operations

After successful authentication and access to protected routes, the user can perform **CRUD (Create, Read, Update, Delete) operations** on their goals:

- **Create**: Add a new goal to the system.
- **Read**: Retrieve and view all goals associated with the user.
- **Update**: Modify goal details, such as title or completion status.

- **Delete**: Remove a goal from the system that is no longer needed.

Each operation is performed only on the goals owned by the authenticated user, ensuring privacy and data integrity.

Project Setup and Configuration

The Micro Goals backend project is organized in a modular and structured way to ensure maintainability, scalability, and clarity. The **folder structure** separates different functionalities such as routing, database models, middleware, and configuration files. This organization follows best practices for Node.js and Express.js applications, making it easier for developers to manage the codebase, add features, and debug issues.

Folder Structure

The project root folder is named **Micro Goals/** and contains subfolders and files organized as follows:

```
Micro Goals/
├── middleware/
│   ├── authMiddleware.js
│   └── errorMiddleware.js

├── Schema/
│   ├── User.js
│   └── GoalSchema.js

├── models/
│   ├── userModel.js
│   └── goalModel.js

├── routes/
│   ├── authRoutes.js
```

```
│   └── goalRoutes.js

├── .env
├── index.js
└── package.json
```

Detailed Explanation of Folders and Files

1. middleware/

This folder contains **middleware functions** that handle tasks such as authentication and error handling.

- **authMiddleware.js**: This middleware checks if a request contains a valid JWT token. It verifies the token and grants access to protected routes only to authenticated users.
- **errorMiddleware.js**: This middleware handles errors across the application. It captures errors thrown in routes or controllers and returns standardized error responses to the client, including appropriate HTTP status codes and messages.

2. Schema/

The Schema folder contains the **Mongoose schemas** that define the structure and validation rules for data stored in MongoDB.

- **User.js**: Defines the schema for user data, including fields like name, email, and password. It may also include pre-save hooks to hash passwords.
- **GoalSchema.js**: Defines the schema for goals, including fields like title, completed, and a reference (userId) linking the goal to a specific user.

3. models/

The models folder contains **Mongoose models** based on the schemas. Models provide the interface to interact with the MongoDB collections.

- **userModel.js**: Represents the User collection and provides methods to create, read, update, or delete user documents.

- **goalModel.js**: Represents the Goal collection and provides methods to manage goal documents in the database.

4. routes/

The routes folder contains **Express route definitions** that handle API endpoints.

- **authRoutes.js**: Contains routes related to user authentication, such as registration and login. These routes use the auth controller to process requests.
- **goalRoutes.js**: Contains routes related to goal management, including creating, updating, deleting, and retrieving goals. Protected routes require authentication via the authMiddleware.

5. .env

The .env file stores **environment variables** such as the MongoDB connection string, JWT secret key, and server port. This allows sensitive information to be kept secure and easily configured without hardcoding values in the codebase.

6. index.js

This is the **entry point of the application**. It initializes the Express server, connects to the MongoDB database, sets up middleware, defines API routes, and starts listening on a specified port.

7. package.json

This file manages **project dependencies** and scripts. It lists all required Node.js packages such as Express, Mongoose, and JWT libraries. It also defines commands for running the server, such as npm start or npm run dev.

Project Setup

Setting up the MicroGoals backend project involves initializing the Node.js environment, installing required dependencies, and starting the server. These steps ensure that the project has all the necessary tools and libraries to run efficiently and securely. The setup process is straightforward and follows standard Node.js and Express.js project practices.

1. Initialize Node.js Project

The first step is to initialize the Node.js project using the command:

npm init -y

This command creates a **package.json** file in the project root directory with default configurations. The package.json file serves as the project manifest, storing metadata about the project, including its name, version, scripts, and dependencies. Initializing the project in this way provides a foundation for managing project dependencies and scripts in a structured manner.

2. Install Dependencies

Next, the required dependencies are installed to provide essential functionality for the backend system. The installation command is:

npm install express mongoose jsonwebtoken bcryptjs dotenv

Each package serves a specific purpose in the MicroGoals project:

- **express**: A lightweight web framework for Node.js used to create server endpoints and manage HTTP requests and responses. It simplifies routing, middleware integration, and server setup.
- **mongoose**: An Object Data Modeling (ODM) library for MongoDB, which provides a schema-based solution to model and interact with the database. It simplifies database operations, schema validation, and relationship management between collections.
- **jsonwebtoken**: A library used to create and verify JWT tokens, which are essential for secure user authentication and access control to protected routes.
- **bcryptjs**: A library for hashing passwords, ensuring that user credentials are securely stored in the database and protected from unauthorized access.
- **dotenv**: A library used to load environment variables from a .env file. This allows sensitive information, such as database credentials and JWT secret keys, to be stored securely and configured easily without hardcoding values in the code.

3. Start the Server

After installing dependencies, the server can be started using the command:

node server.js

or, if using a different entry file like index.js:

node index.js

This command runs the Node.js server, connects to the MongoDB database, and listens for incoming HTTP requests on the specified port. Once the server is running, the backend API is ready to accept requests from clients for user registration, login, and goal management operations.

Backend Development

The backend development of the **Micro Goals** project focuses on building a secure, scalable, and maintainable RESTful API using **Node.js**, **Express.js**, and **MongoDB**. The development process includes setting up the server, connecting to the database, implementing authentication, creating CRUD operations for goals, and adding middleware for security and error handling. Below is a detailed explanation of the backend implementation for documentation purposes.

Created Express Server

An **Express server** was set up to handle HTTP requests and responses. The server is responsible for connecting all API routes, parsing JSON requests, and managing middleware. Express simplifies routing and request handling, enabling the backend to efficiently process operations like user authentication and goal management. By enabling JSON parsing through middleware (express.json()), the server can accept and respond to data in a structured format, which is essential for REST API functionality.

Connected MongoDB

The backend connects to **MongoDB**, a NoSQL database, to store user information and goal data. MongoDB stores data in flexible JSON-like documents, allowing dynamic schemas. The connection is typically established using **Mongoose**, which provides a structured interface for defining schemas, performing validations, and interacting with the database. By connecting MongoDB to the Express server, the application can securely store and retrieve user-specific goals and account details.

Implemented Authentication APIs

The backend includes authentication functionality using **JWT (JSON Web Tokens)**. Two key endpoints were implemented:

- **User Registration**: New users can create accounts by providing their name, email, and password. Passwords are securely hashed using **bcryptjs** before storing them in the database.
- **User Login**: Registered users can log in using their email and password. Upon successful authentication, a **JWT token** is generated and sent to the client. This token is required for accessing protected routes, ensuring secure and user-specific access.

This authentication system ensures that only valid users can access and modify their personal data.

Implemented Goal CRUD APIs

The backend implements **CRUD operations** (Create, Read, Update, Delete) for managing user goals. These APIs follow RESTful architecture principles and enforce **secure, user-specific access** using JWT authentication.

- **Create Goal**: Authenticated users can create new goals. Each goal is associated with the user who created it using a userId reference, ensuring data isolation between users.
- **Read Goal**: Users can retrieve their own goals from the database. The system filters goals by the authenticated user's ID to prevent access to others' data.
- **Update Goal**: Users can update goal attributes, such as marking a goal as completed. Validation ensures that only the owner of the goal can perform updates.
- **Delete Goal**: Users can delete goals they own. Secure route protection prevents users from deleting goals belonging to others.

All CRUD routes are protected using **authentication middleware**, which verifies the JWT token before granting access. If the token is missing or invalid, the system responds with a 401 Unauthorized error.

Here is an example snippet from the goal routes implementation:

```
const express = require("express");
const router = express.Router();
const goalModel = require("../models/goalModel");
const authMiddleware = require("../middleware/authMiddleware");

// Create a new goal
router.post("/create", authMiddleware, async (req, res, next) => {
  try {
    const { title } = req.body;
    const goal = new goalModel({ title, user: req.user });
    await goal.save();
    res.status(200).json({ success: true, data: goal });
  } catch (err) {
    next(err);
  }
});

// Get a specific goal
router.get("/:id", authMiddleware, async (req, res, next) => {
  try {
    const id = req.params.id;
    const data = await goalModel.findOne({ _id: id, user: req.user });
    res.status(200).json({ success: true, data });
  } catch (err) {
    next(err);
```

```
  }
});

// Update a goal
router.put("/:id", authMiddleware, async (req, res, next) => {
  try {
    const goal = await goalModel.findOneAndUpdate(
      { _id: req.params.id, user: req.user },
      { completed: req.body.completed },
      { new: true }
    );
    res.status(200).json({ success: true, message: "Goal updated" });
  } catch (err) {
    next(err);
  }
});

// Delete a goal
router.delete("/:id", authMiddleware, async (req, res, next) => {
  try {
    await goalModel.findOneAndDelete({ _id: req.params.id, user: req.user });
    res.status(200).json({ message: "Goal deleted" });
  } catch (err) {
    next(err);
  }
});
```

```
module.exports = router;
```

Added Authentication Middleware

Authentication middleware is implemented to secure protected routes. The middleware checks for the presence of a valid JWT token in the request header, verifies it, and attaches the authenticated user's information to the request object. This ensures that only logged-in users can access sensitive operations like creating, updating, or deleting goals. Unauthorized requests are blocked, maintaining data security and privacy.

Added Error Middleware

An **error-handling middleware** is added to centralize error management across the application. This middleware captures runtime and validation errors from routes and controllers and returns standardized JSON responses with appropriate HTTP status codes. Common status codes include:

- **200** – Success
- **201** – Resource created successfully
- **400** – Bad request (e.g., missing or invalid data)
- **401** – Unauthorized access
- **404** – Resource not found

Centralized error handling ensures consistent API responses, simplifies debugging, and improves the reliability of the backend system.

Database Development

The **Database Development** section of the MicroGoals project focuses on configuring MongoDB, establishing a connection between the backend server and the database, and ensuring reliable error handling. MongoDB serves as the

primary storage system for user data and goal-related information, enabling efficient and secure data management.

Configure MongoDB

The MicroGoals backend can use either **MongoDB Atlas** (cloud-hosted) or a **local MongoDB** instance.

- **MongoDB Atlas**: A cloud database service that allows remote access and automatic scaling. It is ideal for production environments where secure and reliable access from multiple locations is required.
- **Local MongoDB**: A database installed and hosted on the developer's machine. This is suitable for testing and development purposes.

The database connection requires a **connection string**, which contains the database URI, username, password, and other configuration details. For security and flexibility, this connection string is stored in the **.env file** as an environment variable, typically named MONGO_URI. This approach prevents sensitive credentials from being hardcoded into the application code.

Example .env entry:

MONGO_URI=mongodb+srv://username:password@cluster0.mongodb.net/microgoals?retryWrites=true&w=majority

PORT=5000

JWT_SECRET=your_jwt_secret_key

Create Database Connection

The database connection is established in the **index.js** (or server.js) file using **Mongoose**, which is an ODM (Object Data Modeling) library for MongoDB. Mongoose provides a structured way to define schemas, validate data, and interact with MongoDB collections.

Steps to create the database connection:

1. **Import required modules**:
2. const mongoose = require("mongoose");

3. const dotenv = require("dotenv");
4.
5. dotenv.config();
6. **Connect to MongoDB using mongoose.connect()**:
7. mongoose.connect(process.env.MONGO_URI, {
8. useNewUrlParser: true,
9. useUnifiedTopology: true,
10. })
11. .then(() => console.log("MongoDB connected successfully"))
12. .catch((err) => console.error("MongoDB connection error:", err));
    - useNewUrlParser and useUnifiedTopology are options to ensure compatibility with the latest MongoDB drivers.
    - The then() callback confirms that the connection was successful.
    - The catch() callback handles any connection errors and logs them for debugging.

Handle Connection Errors

Proper **error handling** is critical to ensure the server does not fail silently if the database is unreachable. In addition to the catch() block in mongoose.connect(), developers can also add event listeners for the connection object to monitor errors during runtime:

```
const db = mongoose.connection;


db.on("error", (error) => console.error("MongoDB runtime error:", error));

db.once("open", () => console.log("MongoDB connection established successfully"));
```

- db.on("error") listens for errors after the initial connection and logs them for troubleshooting.

- db.once("open") confirms that the database is ready for operations.

This setup ensures that the backend is aware of any connectivity issues, which is crucial for maintaining reliable API functionality.

Create Schema and Models

In the Micro Goals backend system, **schemas** and **models** are essential for structuring and managing data in MongoDB. Using **Mongoose**, the project defines schemas to specify the shape, types, and validation rules of documents stored in the database. Models are then created from these schemas to provide an interface for performing database operations like creating, reading, updating, and deleting records.

User Schema

The **User Schema** defines the structure of user-related data and ensures that all necessary information is captured in a consistent format. The key fields in the User Schema include:

- **name**: Stores the user's full name. This field is required.
- **email**: Stores the user's email address. It must be unique to prevent duplicate accounts and is required for login.
- **password**: Stores the user's password in a **hashed** form using libraries like bcryptjs for security. Plain-text passwords are never stored in the database.

Example of a User Schema using Mongoose:

```
const mongoose = require("mongoose");

const bcrypt = require("bcryptjs");


const userSchema = new mongoose.Schema({

    name: { type: String, required: true },

    email: { type: String, required: true, unique: true },
```

```
    password: { type: String, required: true },
});

// Hash password before saving
userSchema.pre("save", async function(next) {
    if (!this.isModified("password")) return next();
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
    next();
});

module.exports = mongoose.model("User", userSchema);
```

Key Points:

- Ensures all users have a name, email, and password.
- Enforces **unique email addresses** for account identification.
- Hashes passwords automatically before saving to enhance security.

Goal Schema (with user Id reference)

The **Goal Schema** defines the structure of a goal and includes a reference to the user who owns it. This allows the system to associate each goal with a specific user and enforce user-specific access. The key fields include:

- **title**: The title or description of the goal. Required for creating a new goal.
- **completed**: A boolaen field indicating whether the goal is completed. Defaults to false.
- **User Id**: A reference to the User model (_id field), linking the goal to its owner. This ensures that CRUD operations can be filtered by user, allowing each user to access only their goals.

Example of a Goal Schema using Mongoose:

```
const mongoose = require("mongoose");

const goalSchema = new mongoose.Schema({
   title: { type: String, required: true },
   completed: { type: Boolean, default: false },
   user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
}, { timestamps: true });

module.exports = mongoose.model("Goal", goalSchema);
```

Key Points:

- Allows users to create, update, view, and delete personal goals.
- Uses a **reference (user Id)** to associate each goal with a specific user.
- Supports filtering goals by user in API endpoints to maintain **data privacy and isolation**.
- Includes timestamps for tracking creation and modification dates.

Project Execution

The **Project Execution** section outlines the step-by-step process for running the Micro Goals backend system, testing its functionality, and interacting with the API. These steps ensure that the application is correctly configured, connected to the database, and ready for secure goal management operations.

1. Start MongoDB

Before running the backend server, the MongoDB database must be running. Depending on the setup:

- **Local MongoDB**: Start the MongoDB service on your machine using the terminal or MongoDB Compass.
- **MongoDB Atlas**: Ensure that the cloud database instance is active and accessible, and that the connection string in the .env file is correctly configured.

Starting MongoDB ensures that the backend can store and retrieve user and goal data without errors.

2. Run Server

After confirming that MongoDB is running, start the Express server using Node.js. This can be done with commands such as:

node index.js

or

npm run dev

The server initializes the Express application, connects to MongoDB using Mongoose, sets up middleware, and listens for incoming HTTP requests on the configured port. A successful server start confirms that the backend is ready to process API requests.

3. Test APIs in Thunder Client

Once the server is running, API endpoints can be tested using **Thunder Client**, a REST API client extension for Visual Studio Code. Thunder Client allows sending HTTP requests to the server, including:

- GET requests to fetch data
- POST requests to create new resources
- PUT/PATCH requests to update resources
- DELETE requests to remove resources

Testing APIs in Thunder Client ensures that all endpoints are functioning correctly before integrating the backend with a frontend application.

4. Register User

The first API interaction is **user registration**. Use the registration endpoint to create a new user by providing:

- name
- email
- password

The backend validates the input, hashes the password, and stores the user data in MongoDB. A successful response confirms that the user account has been created.

5. Login and Get JWT

After registration, the user can **log in** using the registered email and password. Upon successful authentication:

- The backend generates a **JWT token**.
- The token is returned to the client and must be included in the Authorization header of all subsequent requests to protected routes.

This JWT token provides secure access control, ensuring that only authenticated users can perform operations on their personal goals.

6. Perform CRUD Operations

With authentication in place, the user can now perform **CRUD operations** on their goals using the protected API endpoints:

- **Create**: Add new goals to the database. Each goal is linked to the authenticated user.
- **Read**: Retrieve all goals associated with the logged-in user. The system filters data based on the user's ID.
- **Update**: Modify the title or completion status of existing goals. Only the owner of the goal can perform updates.

- **Delete**: Remove goals permanently from the system. Users cannot delete goals that belong to others.

All CRUD routes are secured with **authentication middleware**, and any runtime or validation errors are handled centrally using **error middleware**, providing consistent and reliable API responses.

Conclusion

The MicroGoals Secure Goal Management Backend API successfully demonstrates how a modern backend application can be built using Node.js, Express.js, and MongoDB following the Model–View–Controller (MVC) Architecture pattern. The project provides a structured and secure system that allows users to register, log in, and manage their personal goals efficiently through RESTful APIs.

Through the implementation of JSON Web Token (JWT) authentication and middleware, the system ensures that only authorized users can access protected routes and perform operations on their own data. The CRUD functionalities for goals allow users to create, read, update, and delete their personal goals while maintaining proper data security and user isolation.

The project also demonstrates important backend development practices such as centralized error handling, modular folder structure, secure password hashing, and API testing using Thunder Client. These practices improve maintainability, scalability, and reliability of the backend system.

Overall, the MicroGoals project serves as a strong foundation for building secure and scalable backend APIs. It highlights the practical use of modern backend technologies and architectural patterns, making it a valuable learning experience in backend development and REST API design.